3 Methodology

The experiment compares five ways of classifying a clothing image: Linear, MLP, CNN, LSTM and Transformer. Each model receives the same image information, but processes it differently. Linear and MLP use a pixel vector, CNN uses the image grid, LSTM reads image rows, and Transformer works with small image patches. This section describes the implemented models and the procedure used to train and evaluate them.

3.1 Shared data and workflow

Fashion-MNIST contains ten classes of 28×28 grayscale images [1]. As described in Section 2, the official training set is divided into 54,000 training images and 6,000 validation images using a stratified split with seed 42. The official test set contains 10,000 images. Training fits the model weights, validation chooses which saved weights to keep, and testing measures the selected model.

All models use the preprocessing from Section 2. Pixel values are divided by 255 and standardized with the training mean and standard deviation, approximately 0.286209 and 0.353160. The same constants are reused for validation and testing. No images are resized, augmented or removed.

$$\text{Standardized image} \longrightarrow \text{Model-specific input} \longrightarrow \text{Ten class scores}$$

Training updates weights; validation selects weights; testing evaluates them.

Table 1 gives the input to each model. Here, B is the number of images in a batch. Each representation contains all 784 pixel values per image. The labels are integer class indices from 0 to 9. Every model returns a tensor of shape $(B, 10)$ containing raw class scores, called logits.

Table 1: Inputs and trainable parameter counts of the implemented models.

Model	Input shape	Arrangement	Parameters
Linear	$(B, 784)$	One vector per image	7,850
MLP	$(B, 784)$	One vector per image	235,146
CNN	$(B, 1, 28, 28)$	One grayscale image grid	421,642
LSTM	$(B, 28, 28)$	28 rows, each with 28 pixels	82,186
Transformer	$(B, 49, 16)$	49 patches, each with 16 pixels	71,946

3.2 Pixel-based baselines

3.2.1 Linear classifier

The Linear model assigns a weight to each pixel for each class. For a flattened image $x \in \mathbb{R}^{784}$, its output is

$$z = Wx + b, \qquad W \in \mathbb{R}^{10\times 784}, \quad b \in \mathbb{R}^{10}.$$

The largest score determines the prediction. This model provides a small baseline with no hidden layer. It can learn which pixel positions support a class, but cannot learn nonlinear combinations of those pixels.

3.2.2 Multilayer perceptron

The MLP adds two hidden layers with 256 and 128 units. Each hidden layer is followed by ReLU and dropout with probability 0.2. ReLU sets negative values to zero, while dropout randomly suppresses some activations during training.

$$784 \longrightarrow 256 \longrightarrow 128 \longrightarrow 10.$$

The hidden layers allow the MLP to learn nonlinear relationships among pixels. The image is still treated as a vector: nearby pixels do not receive a special shared operation. Dropout is disabled during validation and testing.

3.3 Models for spatial and sequential structure

3.3.1 Convolutional neural network

The CNN keeps the two-dimensional image layout. Convolution applies the same small filter at different image positions, allowing the network to learn local patterns such as edges and contours [2]. Our model has two convolutional blocks, followed by a dense classifier.

Both convolutions use 3×3 kernels, stride 1 and padding 1, so they preserve height and width. ReLU follows each convolution. A 2×2 max-pooling layer then keeps the largest value in each window and halves both spatial dimensions. The first block produces 32 feature maps and the second produces 64.

Table 2: CNN operations and output sizes.

Operation	Output shape
Input image	$(B, 1, 28, 28)$
3×3 convolution, 32 channels; ReLU	$(B, 32, 28, 28)$
2×2 max pooling	$(B, 32, 14, 14)$
3×3 convolution, 64 channels; ReLU	$(B, 64, 14, 14)$
2×2 max pooling	$(B, 64, 7, 7)$
Flatten	$(B, 3136)$
Dense layer, 128 units; ReLU; dropout 0.2	$(B, 128)$
Dense layer, 10 outputs	$(B, 10)$

The final classifier combines the learned spatial features into ten scores. Although convolution shares weights across positions, the dense layer from 3,136 features to 128 units accounts for most of this model's parameters. The CNN therefore has the largest parameter count in this comparison.

3.3.2 Row-sequence LSTM

An LSTM maintains a hidden state and a memory state while reading a sequence [3]. Here, the sequence comes from a static image. The first step receives the top row, the second receives the next row, and the last receives the bottom row. Each step contains 28 pixel values.

For a standardized image X, row t is $r_t = X_{0,t,:}$. The batch shape $(B, 28, 28)$ means *batch, row steps, pixel features*. The two dimensions of size 28 have different roles; the sequence is not a sequence of different images.

$$\text{Row } 1 \longrightarrow \text{Row } 2 \longrightarrow \cdots \longrightarrow \text{Row } 28$$

One shared LSTM reads the rows in order.

$$h_{28} \in \mathbb{R}^{128} \longrightarrow \text{dropout } 0.2 \longrightarrow \text{dense layer} \longrightarrow 10 \text{ scores}$$

The implementation uses one forward LSTM layer with 128 hidden units. Its hidden and memory states start at zero for each image. The gates control what information is retained and updated as successive rows are read. After the 28th row, the final hidden state is passed through dropout and a linear classifier. No state is carried from one image to another.

This design tests whether a compact summary built from ordered rows is useful for classification. It introduces a top-to-bottom reading order, whereas the CNN processes local neighborhoods throughout the image.

3.4 Patch-based Transformer

The Transformer uses attention to combine information from different parts of an image [4]. The input follows the image-patch idea [5], using a small configuration suited to the 28×28 images.

3.4.1 Patch tokens and position information

Each image is divided into a 7×7 grid of non-overlapping 4×4 patches. Each patch is flattened into 16 values. Patches are ordered from left to right across each row of the grid, then from top to bottom. A learned linear layer maps each patch from 16 values to 64 features. A learned positional vector is added so the model can distinguish patch locations:

$$e_j = Wp_j + b + q_j, \qquad j = 1, \ldots, 49,$$

where $p_j \in \mathbb{R}^{16}$ is a patch, $W \in \mathbb{R}^{64\times16}$ is the projection, and $q_j \in \mathbb{R}^{64}$ is its position vector. Dropout with probability 0.2 is applied after this addition.

3.4.2 Attention blocks and classification

Two encoder blocks process the 49 tokens. Each block uses four attention heads, a feature width of 64, and a feedforward network with hidden width 128 and GELU activation. Layer normalization is applied before each attention and feedforward sublayer, with residual connections around both. Dropout is 0.2.

For one attention head, the operation is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{16}}\right)V.$$

The queries Q, keys K and values V are learned projections of the tokens. Each head has 16 features because the 64 features are divided among four heads. Attention weights determine how much information one patch receives from the others. All 49 patches are available to every patch; no causal mask is used.

Table 3: Transformer representation through the network.

Stage	Shape
Flattened image patches	$(B, 49, 16)$
Patch projection and learned positions	$(B, 49, 64)$
Two encoder blocks	$(B, 49, 64)$
Final layer normalization; mean over patches	$(B, 64)$
Dropout 0.2; dense classifier	$(B, 10)$

The final representation is the mean of the 49 normalized output tokens. There is no extra classification token. The encoder internally uses sequence-first tensors of shape $(49, B, 64)$; this transpose changes the layout, not the patch order. All weights are trained from scratch.

3.5 Learning objective

Every model learns to assign a high score to the correct class. For image i, softmax converts the logits $z_{i,k}$ into class probabilities:

$$p_{i,k} = \frac{\exp(z_{i,k})}{\sum_{j=0}^{9}\exp(z_{i,j})}, \qquad \mathcal{L} = -\frac{1}{B}\sum_{i=1}^{B}\log p_{i,y_i}.$$

Here, y_i is the correct class and $\mathcal{L}$ is the average cross-entropy loss. The implementation passes logits directly to `CrossEntropyLoss`, which performs the required calculation. Prediction uses $\hat{y}_i = \arg\max_k z_{i,k}$. Class weights are not used because the training split contains the same number of examples from each class.

3.6 Training procedure

All five models use Adam [6] with learning rate 0.001, batch size 128 and a budget of ten epochs. These settings give each model the same number of passes over the training data. Table 4 records the shared choices.

Table 4: Training and checkpoint selection settings.

Setting	Value
Random seed	42 for Python, NumPy, PyTorch and the training shuffle
Optimizer	Adam, learning rate 0.001
Batch size	128; the smaller last batch is kept
Training duration	10 complete epochs
Loss	Unweighted cross-entropy
Regularization	Dropout 0.2 in MLP, CNN, LSTM and Transformer
Checkpoint selection	Lowest validation loss; an exact tie keeps earlier weights
Data loading	Training shuffled; validation and test ordered; zero workers
Other settings	No augmentation, weight decay, gradient clipping or scheduler

Each model is initialized separately after resetting the seed. A training epoch visits all 54,000 training images. For each batch, the program clears the previous gradients, calculates the logits and loss, backpropagates, and updates the weights with Adam.

At the end of the epoch, the model is evaluated on all 6,000 validation images. Evaluation mode disables dropout, and gradient recording is turned off. The program saves the weights whenever validation loss improves. Training continues through epoch ten, even if the best checkpoint was found earlier. The saved checkpoint with the lowest validation loss is then reloaded for testing.

Epoch loss is weighted by the number of images in each batch, so a smaller final batch does not receive the same weight as a full batch. Accuracy and macro-F1 are calculated from predictions across the complete split. Training metrics are collected while the weights change; validation metrics use the fixed weights at the end of an epoch.

3.7 Evaluation measures

The selected checkpoint is evaluated on all 10,000 official test images. Accuracy measures the fraction classified correctly:

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}(\hat{y}_i = y_i).$$

Here, N is the number of evaluated images and $\mathbf{1}$ is one when the prediction is correct, otherwise zero.

Macro-F1 gives equal weight to every class. For class k, true positives TP_k are correct predictions of that class, false positives FP_k are other classes predicted as k, and false negatives FN_k are examples of k predicted as another class:

$$F1_k = \frac{2TP_k}{2TP_k + FP_k + FN_k}, \qquad \text{MacroF1} = \frac{1}{10}\sum_{k=0}^{9} F1_k.$$

Undefined class scores are assigned zero. Both accuracy and macro-F1 are reported because overall accuracy alone does not show how performance varies among categories.

The evaluation also saves cross-entropy loss, per-class precision and recall, and a confusion matrix. Matrix rows represent true labels and columns represent predicted labels. Correct and incorrect example images help explain recurring mistakes. These outputs support the results and error analysis; they do not select the training checkpoint.

3.8 Runtime, reproducibility and comparison scope

3.8.1 Runtime measurements

Training time covers the ten-epoch training and validation loop, including saving improved checkpoints. It excludes data preparation before the loop and the final checkpoint reload and validation after it.

Forward-pass timing uses a fixed batch of 128 images already on the device. After ten warm-up calls, the program times 100 forward calls and averages their duration. Dividing the batch time by 128 gives time per image at that batch size. Device synchronization is performed around the timed region. This measurement excludes data loading and host-to-device transfers and describes batched throughput, not the latency of a single-image request.

A separate timer measures the complete test pass, including loading, transfers, model execution, loss and prediction collection. It excludes later metric summaries and plots. The two measurements are reported separately because they cover different work.

3.8.2 Reproducibility

The implementation separates data preparation in `a1/data.py`, model definitions in `a1/models.py`, training in `a1/train.py`, and testing in `a1/evaluate.py`. Configuration files specify the shared training settings. The split cache records the training and validation indices and the normalization constants.

Each run saves the selected checkpoint, epoch history and a summary of its configuration and environment. Evaluation checks the recorded data and split hashes and normalization values before using the checkpoint. It also saves per-image predictions so that the reported scores can be checked again.

The following commands illustrate the CNN workflow from the `A1` folder with the project dependencies installed. The other models use the same entry points and their corresponding configuration files.

```
python -m a1.train -model cnn -config configs/cnn.json
python -m a1.evaluate -checkpoint runs/m2/cnn/best.pt -device cuda
```

3.8.3 What the comparison can establish

The split, preprocessing, seed, optimizer, learning rate, batch size and epoch budget are shared. The architecture, input arrangement and parameter count change. The experiment therefore compares these five implemented configurations. It does not hold model capacity or computation constant.

The saved Linear and MLP runs used MPS, while CNN, LSTM and Transformer training used CUDA. The saved test evaluations used an NVIDIA GeForce GTX 1660 SUPER. Training times must retain their hardware labels and cannot support a direct speed ranking across the two platforms. Finally, each model has one recorded training seed. Differences in the resulting scores describe these runs; repeated seeds would be needed to measure how much the results vary.

References

[1] H. Xiao, K. Rasul and R. Vollgraf. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. 2017. arXiv:1708.07747.

[2] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner. "Gradient-Based Learning Applied to Document Recognition." *Proceedings of the IEEE*, 86(11), 2278–2324, 1998. Author's publication page.

[3] S. Hochreiter and J. Schmidhuber. "Long Short-Term Memory." *Neural Computation*, 9(8), 1735–1780, 1997. doi:10.1162/neco.1997.9.8.1735.

[4] A. Vaswani et al. *Attention Is All You Need*. 2017. arXiv:1706.03762.

[5] A. Dosovitskiy et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. 2020. arXiv:2010.11929.

[6] D. P. Kingma and J. Ba. *Adam: A Method for Stochastic Optimization*. 2015. arXiv:1412.6980.